arXiv:2401.02412v1 [cs.LG] 4 Jan 2024

LLM AUGMENTED LLMS: EXPANDING CAPABILITIES THROUGH COMPOSITION

Rachit Bansal[1] **Bidisha Samanta**[1] **Siddharth Dalmia**[2] **Nitish Gupta**[1] **Shikhar Vashishth**[1]
Sriram Ganapathy[1] **Abhishek Bapna**[1] **Prateek Jain**[1] **Partha Talukdar**[1]
[1]Google Research [2]Google DeepMind

ABSTRACT

Foundational models with billions of parameters which have been trained on large corpora of data have demonstrated non-trivial skills in a variety of domains. However, due to their monolithic structure, it is challenging and expensive to augment them or impart new skills. On the other hand, due to their adaptation abilities, several new instances of these models are being trained towards new domains and tasks. In this work, we study the problem of efficient and practical composition of existing foundation models with more specific models to enable newer capabilities. To this end, we propose CALM—Composition to Augment Language Models—which introduces cross-attention between models to compose their representations and enable new capabilities. Salient features of CALM are: (i) Scales up LLMs on new tasks by 're-using' existing LLMs along with a few additional parameters and data, (ii) Existing model weights are kept intact, and hence preserves existing capabilities, and (iii) Applies to diverse domains and settings. We illustrate that augmenting PaLM2-S with a smaller model trained on low-resource languages results in an absolute improvement of up to 13% on tasks like translation into English and arithmetic reasoning for low-resource languages. Similarly, when PaLM2-S is augmented with a code-specific model, we see a relative improvement of 40% over the base model for code generation and explanation tasks—on-par with fully fine-tuned counterparts.

1 INTRODUCTION

Large Language Models (LLMs) have shown to encompass a range of foundational capabilities such as commonsense and factual reasoning, world knowledge, and coherent language generation (Bubeck et al., 2023; Google et al., 2023). Leveraging these foundational capabilities, a number of efforts in the community have fine-tuned these models to enable domain-specific capabilities such as code generation, copy editing, and mathematical problem solving (Lewkowycz et al., 2022; Singhal et al., 2023). This has resulted in the development of several specialized large models with domain-specific capabilities. For example, there are models that do well on standard code generation but are not as proficient in general logical reasoning and vice-versa. Presence of such a large number of domain-specific models leads to a natural question: Can we compose an *anchor* model with a domain-specific *augmenting* model to enable new capabilities? For example, can we compose an augmenting model's code understanding capability with an anchor LLM's language generation capability to enable code-to-text generation capability?

The typical approach for this problem is to further pre-train or (efficiently) fine-tune the anchor model on the data that was originally used to train the augmenting model (Hu et al., 2022; Kessler et al., 2021). However, many a times such solutions are not feasible since training large models is computationally expensive, especially since the augmenting model itself may be an LLM trained on a massive corpora. Further, processing data from multiple sources might not be feasible due to privacy concerns and organizational boundaries. Working with multiple distinct models is also desirable since it allows the reuse of existing models with established capabilities, providing better control and avoiding catastrophic forgetting that is prevalent in conventional approaches.

Correspondence to Rachit and Bidisha: [brachit, bidishasamanta]@google.com

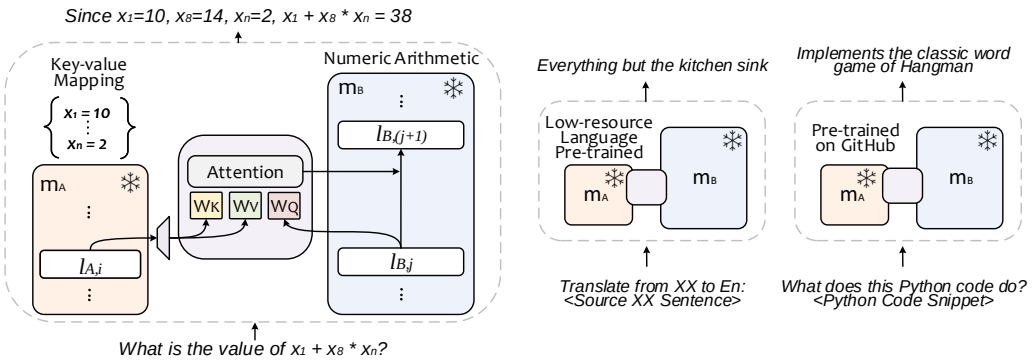

Figure 1: **Overview of CALM.** To augment an *anchor* LLM ($\mathbf{m}_\text{B}$) with new capabilities through *composition* with a specialized *augmenting* model ($\mathbf{m}_\text{A}$). Figure illustrates three $\mathbf{m}_\text{A}$ with different capabilities: key-value mapping (*left*), low-resource languages (*center*), and code (*right*). Models $\mathbf{m}_\text{A}$ and $\mathbf{m}_\text{B}$ remain unchanged (❄) during composition. A few additional parameters are learnt over models' layer representations. Leftmost plot shows an $\mathbf{m}_\text{A}$ trained on a set of string-integer mappings, e.g., $\{x_1 : 10, \ldots, x_n : 2\}$. $\mathbf{m}_\text{B}$ is a large LM with arithmetic capabilities. CALM composes these two frozen models to solve the task of arithmetic on keys which either models could not solve on their own (§4.1). Notably, CALM generalizes to the entire key-value set despite training with arithmetic examples spanning only 20% of the keys.

To address the training and the data challenges mentioned above, we propose and study a practical setting for *model composition*: (i) we are given access to one (or more) augmenting model(s) and an anchor model, (ii) we are *not allowed* to modify the weights of either models, and (iii) we only have access to a small amount of data, representing the "combined skills" of the given models, e.g., code generation with complex logical reasoning.

Prior work has largely approached the question of composition from either a routing or a merging standpoint, neither of which provide an effective solution to capture this setting. Routing between the given models, i.e., choosing an output of one model over the other (Ma et al., 2019), or performing a soft ensemble (Muqeeth et al., 2023) is not effective when neither of the models can demonstrate the desired capability. Another body of work creates a combined model by an arithmetic combination of base model parameters (Wortsman et al., 2022; Ilharco et al., 2022; Matena & Raffel, 2022). However, these settings are naturally restrictive and their efficacy is unclear when combining models with different sizes and pre-training objectives (Yadav et al., 2023).

In this work, we propose a novel **Composition to Augment Language Models** (**CALM**) framework to address the general model composition setting mentioned above. Rather than a shallow combination of the augmenting and anchor LMs (Wortsman et al., 2022; Ilharco et al., 2022), CALM introduces a small number of trainable parameters over both augmenting and anchor models' intermediate layer representations. CALM finds an effective combination of the given models to perform new challenging tasks more accurately than either of the models alone, while preserving the capabilities of individual models. Figure 1 highlights few motivating scenarios for CALM.

We study key practical applications of CALM: language inclusivity and code generation. For language inclusivity (§4.2), we use a model that has been trained on a set of low-resource languages. We observe that composing this model with the LLM allows us to borrow its generation and reasoning capabilities to achieve significantly better performance on translation and arithmetic reasoning tasks for low-resource languages (Tables 2 and 3). This composed model outperforms not only the two base models but also versions of the LLM that have been further pre-trained or LoRA (Hu et al., 2022) fine-tuned for the set of low-resource languages. For code generation (§4.3), we use a model that has been trained on open-source code across a variety of programming languages. Composing this model with the LLM—hence borrowing its low-level logic and generation capabilities—outperforms the two base models (Table 4) on code explanation and code completion tasks.

2 Related Works

Parameter efficient fine-tuning: A large body of work focuses on efficient ways of fine-tuning models for new domains by introducing a small number of trainable parameters, keeping the original model intact (Houlsby et al., 2019; Wang et al., 2021; Pfeiffer et al., 2021; Hu et al., 2022; Kessler et al., 2021). Since this paradigm allows a small set of new parameters to be trained, it is challenging to use this approach to adapt a model to a new domain, which is absent from the original training corpus. In contrast, CALM enables a model to be adapted to completely new domains using an augmenting model. In Section 4.4, we demonstrate that CALM is significantly more effective than LoRA (Hu et al., 2022), a representative parameter efficient fine-tuning method.

Model Merging: Merging different expert models with simple techniques like task vector averaging provides a way of recombining different capabilities of these models (Ilharco et al., 2022; Matena & Raffel, 2022). However, these methods are only relevant when the original models are well aligned. Other related approaches are also applicable only when the models are derived from the same model (Matena & Raffel, 2022) or they are of same size (Muqeeth et al., 2023). In contrast, CALM is more generic and is applicable to any set of models.

Model and Task Compositionality: The modular encoder-decoder based method in (Dalmia et al., 2022) adapts components of encoder-decoder models to allow flexible re-usability of different encoders, each with their own capabilities. Several past studies explore compositionality from a multi-modal standpoint. Alayrac et al. (2022) introduce cross-attention parameters across a language model in order to attend to representations coming from an image encoder. They show very effective transfer of capabilities between the two models. In this work, we extend the ideology of model re-use and modularity to extend composition of capabilities in a large language model.

Models as Tools: Another interesting direction for using multiple language models to solve a downstream task has been to perform composition in the models' input text space (Zeng et al., 2022; Shen et al., 2023). Schick et al. (2023) have demonstrated how a model can be taught to use external tools—there might be an opportunity to investigate if other models can be called as a part of the same framework. Since these approaches require a large amount of prompt engineering, in this work we focus on composition through representations that can be learnt automatically.

3 Composition to Augment Language Models (CALM)

Given an ***anchor model*** $\mathbf{m}_{\text{B}}$ and an ***augmenting model*** $\mathbf{m}_{\text{A}}$, CALM aims to compose the two models ($\mathbf{m}_{\text{A}\oplus\text{B}}$) to enable new capabilities as a composition of capabilities of the two individual models.

As discussed in the introduction, we study this composition in a practical setting with the following assumptions: i) we can access weights, run forward and backward pass, and access intermediate representations of both $\mathbf{m}_{\text{B}}$ and $\mathbf{m}_{\text{A}}$, ii) we are not allowed to change weights of both the models, iii) we do not have access to the training data, hyperparameters, training states of both the base models, iv) we are provided a few examples from the target composition domain.

The goal is to learn a composition $\mathbf{m}_{\text{A}\oplus\text{B}} = f(\mathbf{m}_{\text{A}}, \mathbf{m}_{\text{B}}, \Theta_{\mathbf{C}}, \mathbf{D}_{\mathbf{C}})$ to achieve some joint task C. The weights of $\mathbf{m}_{\text{A}}$ and $\mathbf{m}_{\text{B}}$ are frozen. $\Theta_{\mathbf{C}}$ is the additional set of trainable parameters introduced to learn the composition and $\mathbf{D}_{\mathbf{C}}$ refers to the set of examples that are used to learn this composition.

3.1 Learning to Compose ($\Theta_{\mathbf{C}}$)

As outlined in Figure 1, we operate over a selected set of layers from $\mathbf{m}_{\text{B}}$ and $\mathbf{m}_{\text{A}}$ at all times. We learn two sets of additional parameters over these layers: (i) A simple set of linear transformations, $f_{\text{proj}}(.)$ that maps an ith layer representation from $\mathbf{m}_{\text{A}}$ to the dimensionality of representations from $\mathbf{m}_{\text{B}}$, and (ii) A set of cross-attention layers, $f_{\text{cross}}(.,.)$ that cross-attend between this transformed layer representation and a jth layer representation from $\mathbf{m}_{\text{B}}$.

Compositional Layers: Let the augmenting model $\mathbf{m}_{\text{A}}$ and the anchor model $\mathbf{m}_{\text{B}}$ have N_A and N_B layers, respectively. Also, let D_A and D_B be the token dimensionality of the two models. We first choose a set of *compositional* layers—$\mathbb{L}_A$ and $\mathbb{L}_B$—for both models, over which the set of new

learnable parameters are introduced during composition. $n_A = |\mathbb{L}_A|$ and $n_B = |\mathbb{L}_B|$. For simplicity, we set $n_A = n_B = n$ and the gap between two contiguous selected layers is kept uniform based on the number of selected layers—that is, $(l_2 - l_1) = \cdots = (l_n - l_{(n-1)}) = N/n$. Further, $\mathbb{H}_A \in \{\mathbf{H}_{\text{A}1}, \mathbf{H}_{\text{A}2}, \ldots, \mathbf{H}_{\text{A}n_A}\}$ denote the layer representation of a given input after each layer in $\mathbb{L}_A$.

Learned Projections: Next we map representations from $\mathbf{m}_{\text{A}}$ to that of $\mathbf{m}_{\text{B}}$ via a projection layer. In particular, for each layer in $\mathbb{L}_A$, we learn a projection function $f_{\text{proj}} : \mathbb{R}^{D_A} \rightarrow \mathbb{R}^{D_B}$, that projects representations from these layers to the desired representation size of $\mathbf{m}_{\text{B}}$. Let,

$$f_{\text{proj}}(\mathbb{H}_A) \leftarrow \{f_{\text{proj}}(\mathbf{H}_{\text{A}1}), f_{\text{proj}}(\mathbf{H}_{\text{A}2}), \ldots, f_{\text{proj}}(\mathbf{H}_{\text{A}n_A})\}$$

This transformation enables cross-attention across models, and also performs an alignment of representations from $\mathbf{m}_{\text{A}}$ and $\mathbf{m}_{\text{B}}$ despite frozen weights of the base models.

Cross-attention Layers: Similar to the multi-headed cross-attention in encoder-decoder models (for example Vaswani et al. (2017) and Raffel et al. (2020))—we introduce cross-attention between representations of the anchor and the augmenting model. In particular, we use $f_{\text{proj}}(\mathbf{H}_{\text{A}i})$ from the augmenting model as the *key* and *value* vectors for each head in cross-attention. We use the vector $\mathbf{H}_{\text{B}j}$ from the anchor model as the *query* vector, which leads to the following cross-attention setup:

$$\begin{aligned} f_{\text{cross}}(f_{\text{proj}}(\mathbf{H}_{\text{A}i}), \mathbf{H}_{\text{B}j}) &= \text{Concat.}_k\left(\text{head}_k\right)\mathbf{W}^O \quad \forall k \in N_H \\ \text{where, head}_k &= \text{Attn.}(\mathbf{Q}_B, \mathbf{K}_A, \mathbf{V}_A), \\ \text{and, } \mathbf{Q}_B &= \mathbf{H}_{\text{B}j}\mathbf{W}_k^Q, \\ \mathbf{K}_A, \mathbf{V}_A &= f_{\text{proj}}(\mathbf{H}_{\text{A}i})\mathbf{W}_k^K,\ f_{\text{proj}}(\mathbf{H}_{\text{A}i})\mathbf{W}_k^V \end{aligned}$$

Here, N_H represents the number of attention heads used for cross-attention which, in our case, is typically the same as the number of heads used for self-attention in $\mathbf{m}_{\text{B}}$. Each of $\mathbf{W}^O \in \mathbb{R}^{D_B \times D_B}$, $\mathbf{W}_k^Q$, $\mathbf{W}_k^K$, and $\mathbf{W}_k^V \in \mathbb{R}^{D_B \times D_B//N_H}$ are learnable weight matrices, where $k \in \{1..N_H\}$.

Finally, the cross-attention output is added as a residual connection to the layer representations of $\mathbf{m}_{\text{B}}$. The resultant output vector, in-turn, is the input to the succeeding layer in $\mathbf{m}_{\text{B}}$:

$$\mathbf{H}_{\text{A}\oplus Bj} = \mathbf{H}_{\text{B}j} + f_{\text{cross}}(f_{\text{proj}}(\mathbf{H}_{\text{A}i}), \mathbf{H}_{\text{B}j})$$

Here, $\mathbf{H}_{\text{A}\oplus Bj}$ denotes the input to the $(j+1)^{th}$ layer of the composed model. All layers in $\mathbb{L}_A$ and $\mathbb{L}_B$ are utilized in a similar manner. Propagating over the remaining layers in $\mathbf{m}_{\text{B}}$ gives us a final output token y_t decoded for the tth timestep. Akin to usual auto-regressive decoding, the output token for each time-step is appended to the input: $x_{t+1} = x_t \oplus y_t$, Since the updated input at each time step is passed to both models, all representations for the two models are refreshed.

3.2 Composition Training Data ($\mathbf{D_C}$)

Since the target model $\mathbf{m}_{\text{A}\oplus\text{B}}$ involves a composition over the two models $\mathbf{m}_{\text{A}}$ and $\mathbf{m}_{\text{B}}$, we construct the set of training examples $\mathbf{D_C}$ to depict a "combined skill" that enables $\Theta_{\mathbf{C}}$ to attend over the two models appropriately for the target task.

Ideally, if the set of tasks involved in composition task are distinguished as $\mathbf{t}_1$ and $\mathbf{t}_2$ respectively, then we design $\mathbf{D_C}$ to depict the a joint task $\mathbf{C}$. For example, with respect to our synthetic key-value setup: our final task ($\mathbf{C}$) is to perform arithmetic over a set of keys. The augmenting model $\mathbf{m}_{\text{A}}$ is trained to learn the given key-value pairs (notated as task, $\mathbf{t}_1$) and the anchor model $\mathbf{m}_{\text{B}}$ is generic model that can perform numeric arithmetic well (task $\mathbf{t}_2$). For learning the set of parameters $\Theta_{\mathbf{C}}$ for composition, we consider $\mathbf{D_C}$ to be arithmetic over a held-in set of keys (task $\mathbf{C}$), encompassing combined skills from the two models. In contrast to fine-tuning approaches like LoRA (Hu et al., 2022) that would require the entire knowledge source (here, key-values) during training time, we find that training composition on only a fraction of the keys can generalize to the full set.

In other real world settings, a clear distinction in specializing tasks for each model might be difficult to formulate and hence defining a task that captures the combined skills can be challenging. We find that using a set of examples that capture certain capabilities of the two models suffices, i.e., some rough notion of $\mathbf{t}_{A\cup B}$. For our language inclusivity task, we use a mixture of examples containing a small amount of low-resource language and high-resource language data.

Composing multiple models: Finally, we note that while the method has been presented for a setting with one anchor model and only one augmenting model, CALM is applicable to multiple augmenting models as well. In particular, CALM would require learning similar projection and cross-attention components between the anchor and each of the augmenting model. We leave a thorough investigation of this as a topic of future work.

4 EXPERIMENTS

We demonstrate the following in three domains: **(a)** an anchor LLM ($\mathbf{m}_{\text{B}}$) can be composed with an augmenting model ($\mathbf{m}_{\text{A}}$) trained on mappings between string keys and number values to solve arithmetic expressions over those keys requiring both, knowledge of the KV mappings and arithmetic capabilities (§4.1); **(b)** how CALM can be used to expand the language coverage of an anchor LLM ($\mathbf{m}_{\text{B}}$) to low-resource languages it has not seen during pre-training. We show that an augmenting model ($\mathbf{m}_{\text{A}}$) pre-trained on low-resource languages can be composed with such an anchor model to significantly improve translation and math-word problem solving capabilities in low-resource languages (§4.2); **(c)** how code completion and explanation can be improved by composing an anchor LLM with an augmenting model ($\mathbf{m}_{\text{A}}$) specializing in the code domain (§4.3).

In all experiments, we start with a PaLM2-XXS model and further train it on domain-specific data to arrive at an augmenting model ($\mathbf{m}_{\text{A}}$) that is then kept frozen during composition. Note that no task specific training data was used to train CALM. We use PaLM2-XS or PaLM2-S models as the anchor LLM ($\mathbf{m}_{\text{B}}$) that is also kept frozen during composition training. For all our experiments, we set $N_A/n = 4$, i.e., we perform composition using every 4th layer output from $\mathbf{m}_{\text{A}}$. Correspondingly, layers from $\mathbf{m}_{\text{A}}$ ($\mathbb{L}_B$) are chosen such that $n_B = n_A = n$, hence $n_B = N_A/4$.

4.1 KEY-VALUE ARITHMETIC

We first study the setting where we have a small augmenting LM that has been trained to memorize string-to-integer key-value (KV) mappings, and a large anchor LM that is capable of performing arithmetic over integers. We wish to use CALM to compose them and enable a new capability of solving arithmetic expressions containing those keys.

Key-Value Domain Knowledge We first generate a repository of KV pairs containing $\text{N}_{\text{KV}} = 25\text{K}$ pairs by sampling English strings of length $2-6$ characters from the vocabulary of the PaLM2-XXS model and randomly assigning them unique integer values in the range $[1, \text{N}_{\text{KV}}]$. This constitutes the knowledge artifact, $\mathbf{D}_{\text{KV}}$. We further generate a collection of arithmetic expressions ($\mathbf{D}_{\text{KV-EXP}}$) containing addition ($+$), subtraction ($-$), and multiplication ($\times$) operations between $3-6$ keys by randomly sampling keys from $\mathbf{D}_{\text{KV}}$ and operations to perform between them.

Using these arithmetic expressions, we generate three datasets:

(i) KV-Substitution ($\mathbf{D}_{\text{KV-SUBS}}$): This dataset maps each expression in $\mathbf{D}_{\text{KV-EXP}}$, to an expression where the keys are replaced by their corresponding values. For example, this dataset contains examples of the form (<K1> + <K2> − <K3>, $10 + 22 - 24$).

(ii) KV-Arithmetic ($\mathbf{D}_{\text{KV-MATH}}$): This dataset maps each expression in $\mathbf{D}_{\text{KV-EXP}}$ to the numeric value arrived at by solving the arithmetic expression when the keys would be replaced by the corresponding values. For example, examples in this dataset look like (<K1> + <K2> − <K3>, 8).

(iii) Numeric-Arithmetic ($\mathbf{D}_{\text{NUM-MATH}}$): This dataset maps the value substituted version of each expression in $\mathbf{D}_{\text{KV-EXP}}$ to the numeric value arrived at by solving the arithmetic expression. For example, examples in this dataset look like ($10 + 22 - 24$, 8).

Models We obtain augmenting model $\mathbf{m}_{\text{A}}$ by further training a pre-trained PaLM2-XXS model on $\mathbf{D}_{\text{KV-SUBS}}$ to make it memorize the KV pairs in $\mathbf{D}_{\text{KV}}$. Note that, training on $\mathbf{D}_{\text{KV-SUBS}}$ does not teach this augmenting model how to solve arithmetic expressions. Next, we use a pre-trained PaLM2-XS model as the anchor model $\mathbf{m}_{\text{B}}$. This model is capable of solving numeric expressions with decent performance (see Table 1). Note that, this model has no knowledge of the KV pairs in $\mathbf{D}_{\text{KV}}$.

We now take examples from the KV-Substitution dataset $\mathbf{D}_{\text{KV-SUBS}}$ that only span 20% of the keys in $\mathbf{D}_{\text{KV}}$ to form the training data for composition ($\mathbf{D}_{\mathbf{C}}$). We use $\mathbf{D}_{\mathbf{C}}$ to compose the augmenting model

($\mathbf{m}_A$) having knowledge of $\mathbf{D}_{KV}$ and the pre-trained anchor model $\mathbf{m}_B$ by training the composition parameters (Θ_C) using CALM as explained in §3. Both $\mathbf{m}_A$ and $\mathbf{m}_B$ are kept unchanged.

Evaluation Task We evaluate the composed model $\mathbf{m}_{A \oplus B}$ for its ability to solve arithmetic expressions containing keys from $\mathbf{D}_{KV}$. Specifically, we evaluate on the subset of $\mathbf{D}_{KV\text{-}MATH}$ dataset that does not contain expressions used in $\mathbf{D}_C$ during training. This way, we are able to measure the composed model's ability to generalize to keys beyond what was observed during training.

Results Table 1 shows the performance of the three models: $\mathbf{m}_A$, $\mathbf{m}_B$, and $\mathbf{m}_{A \oplus B}$ across the aforementioned datasets. First, we observe that the augmenting model $\mathbf{m}_A$ achieves 98.1% at the KV-Substitution task showing that memorizes $\mathbf{D}_{KV}$ well. Next, we see that it performs poorly (4.2%) at the Numeric-Arithmetic task showing that it does not have arithmetic capabilities. As a result, this model is not able to solve arithmetic expressions containing keys from $\mathbf{D}_{KV}$.

As expected, the anchor model $\mathbf{m}_B$ gets 0% accuracy on the KV-Substitution and KV-Arithmetic tasks as it has not seen any data from $\mathbf{D}_{KV}$. However, it performs well (73.7%) on the Numeric-Arithmetic task demonstrating capability of arithmetic over numerals.

	$\mathbf{m}_A$	$\mathbf{m}_B$	CALM ($\mathbf{m}_{A \oplus B}$)
$\mathbf{D}_{KV\text{-}SUBS}$	98.1	0.0	92.9
$\mathbf{D}_{NUM\text{-}MATH}$	4.2	73.7	72.0
$\mathbf{D}_{KV\text{-}MATH}$	0.7	0.0	**84.3**

Table 1: Evaluation (accuracy (%)) for a synthetic key-value (KV) task. $\mathbf{m}_A$ is trained to memorize the KV mappings while $\mathbf{m}_B$ excels at arithmetic We see that a composition $\mathbf{m}_{A \oplus B}$ is able to perform arithmetic over held-out keys.

Lastly, we see that the composed model $\mathbf{m}_{A \oplus B}$ is able to solve all tasks with high accuracy, especially the KV-Arithmetic task (84.3%) which both the underlying models fail at. This shows that the composed model is able to leverage the relevant capabilities from both the augmenting and anchor model to solve a complex task.

4.2 Low-resource Language Inclusivity

Model	FLORES-200 (XX to En; chrF1)										
	lij	mr	taq	nn	su	ban	pl	th	min	acm	*avg.*
PaLM2-XXS	24.0	16.5	21.6	33.3	20.6	2.1	5.3	63.2	44.0	59.8	29.0
+ NTL ($\mathbf{m}_A$)	32.0	21.6	46.9	50.0	40.6	4.1	4.0	63.8	47.8	61.1	37.2
PaLM2-S ($\mathbf{m}_B$)	32.6	24.2	44.6	50.8	50.9	5.4	9.5	69.0	61.0	68.6	41.7
CALM ($\mathbf{m}_{A \oplus B}$)	**44.1**	**30.4**	**55.1**	**54.6**	**54.4**	**11.8**	**11.3**	**69.4**	**61.1**	**68.9**	**46.1**
$\mathbf{m}_B$+NTL ($\mathbf{m}_B^{NTL}$)	48.1	39.1	59.2	57.5	57.3	11.4	9.9	69.4	61.4	69.0	48.2

Table 2: Translation performance for XX to English direction on the FLORES-200 dataset (Costa-jussà et al., 2022): We show results for a subset of 10 low-resource languages. Note that the composed model $\mathbf{m}_{A \oplus B}$ significantly outperforms both $\mathbf{m}_A$ and $\mathbf{m}_B$. On the complete language list, $\mathbf{m}_{A \oplus B}$ outperforms both the underlying models for 175 of 192 languages (Appendix A; Figure 2). $\mathbf{m}_B^{NTL}$ represents a skyline where $\mathbf{m}_B$ has been further pre-trained on $\mathbf{D}_{NTL}$. The composed model achieves similar performance for a tiny fraction of the training cost.

In this section, we study if we can compose such a large anchor LM $\mathbf{m}_B$ with a smaller augmenting LM $\mathbf{m}_A$ that has been pre-trained on low-resource languages, to perform translation and math-word problem solving tasks presented in these low-resource languages.

Low-resource Language Corpora We use the long-tail language set and the associated corpora from the Next Thousand Languages (NTL) effort (Caswell et al., 2020; Bapna et al., 2022) as the domain data $\mathbf{D}_{NTL}$. This large-scale corpora contains web-crawled monolingual sentences and translation pairs for ∼1000 languages. The dataset has been used for language expansion in translation systems and language models (Garcia et al., 2021; Siddhant et al., 2022).

Model	GSM8K (Low-resource Languages; Accuracy)										
	meo	mfa	pcm	efi	min	ilo	ady	mai	nso	mzn	*avg.*
PaLM2-XXS	5.2	6.8	6.8	4.0	5.6	7.2	6.0	3.6	7.2	6.8	5.9
+ NTL ($\mathbf{m}_{\text{A}}$)	7.6	4.0	4.4	3.2	6.0	4.8	6.4	3.2	6.0	4.8	5.0
PaLM2-S ($\mathbf{m}_{\text{B}}$)	28.8	14.0	**34.4**	14.8	**25.2**	14.8	30.0	22.8	8.4	31.6	22.5
CALM ($\mathbf{m}_{\text{A}\oplus\text{B}}$)	**34.0**	17.6	33.6	**18.0**	23.6	**16.8**	**36.4**	**24.8**	8.4	**36.4**	**25.0**
$\mathbf{m}_{\text{B}}^{\text{NTL}}$	33.2	20.4	31.6	14.0	24.8	14.0	29.2	21.2	9.6	27.6	22.6

Model	(High-resource Languages)										
	en	te	bn	sw	ja	zh	th	fr	es	de	*avg.*
PaLM2-XXS	5.6	4.0	2.0	7.6	2.0	4.4	6.0	6.8	5.6	9.2	5.3
+ NTL ($\mathbf{m}_{\text{A}}$)	4.8	3.6	3.2	4.8	**3.2**	7.6	6.4	9.2	5.6	7.2	5.6
PaLM2-S ($\mathbf{m}_{\text{B}}$)	36.8	19.2	23.2	16.0	2.0	39.2	29.6	38.0	32.4	43.2	28.0
CALM ($\mathbf{m}_{\text{A}\oplus\text{B}}$)	**37.2**	**28.0**	**27.2**	**18.0**	2.4	**43.6**	**33.2**	**42.8**	**36.0**	**49.2**	**31.8**
$\mathbf{m}_{\text{B}}^{\text{NTL}}$	36.0	17.6	18.4	14.4	0.8	33.6	27.2	34.8	31.2	42.0	25.6

Table 3: Evaluations for grade-school mathematics (GSM) problems on low-resource (LRL) and high-resource (HRL) languages. We observe that CALM yields significant gains for both evaluation sets. Gains on the HRL set suggests that CALM avoids catastrophic forgetting.

Models Akin to §4.1, we obtain augmenting model $\mathbf{m}_{\text{A}}$ by training the PaLM2-XXS model on $\mathbf{D}_{\text{NTL}}$ to impart knowledge about these low-resource languages to the model. For $\mathbf{m}_{\text{B}}$, we use the pre-trained PaLM2-S model. We use $\sim 5\%$ of the same low-resource language corpora $\mathbf{D}_{\text{NTL}}$ as the training data $\mathbf{D}_{\mathbf{C}}$ to compose $\mathbf{m}_{\text{A}}$ and $\mathbf{m}_{\text{B}}$ via CALM. Since both models are untrained during composition, the anchor model $\mathbf{m}_{\text{B}}$ is *not* trained on any of the low-resource language data.

Evaluation Tasks We evaluate the composed model $\mathbf{m}_{\text{A}\oplus\text{B}}$ on two tasks:

(i) Translating text from a non-English language to English: We carry out these evaluations in a 5-shot in-context learning paradigm on the FLORES-200 (Costa-jussà et al., 2022) dataset. This dataset contains examples for 200 high- and low-resource languages.

(ii) Performing grade school math word problems expressed in a non-English language: We evaluate on the multilingual version of the GSM-8K dataset (Shi et al., 2023) containing math word problems for English and 9 other high-resource languages. We further generated a silver-standard GSM-8K dataset for low-resource languages by automatically translating the English examples in GSM-8K to 25 low-resource languages supported by Google Translate.[1]

Results Table 2 shows results on the FLORES-200 dataset (Costa-jussà et al., 2022), where the input is a low-resource (XX) language sentence and the output should be the corresponding English translation. For 10 low-resource languages shown in the Table, we see that both the underlying models $\mathbf{m}_{\text{A}}$ and $\mathbf{m}_{\text{B}}$ are outperformed by our composed model $\mathbf{m}_{\text{A}\oplus\text{B}}$. We find that the composed model $\mathbf{m}_{\text{A}\oplus\text{B}}$ outperforms $\mathbf{m}_{\text{B}}$ on 175 of the complete set of 192 languages (Appendix A).

Table 3 shows the performance of these models on the grade-school math word problems from the GSM8K task (Cobbe et al., 2021) on low-resource languages (*top*) and high-resource languages (Shi et al. (2023); *bottom*). Firstly, we observe that the augmenting model $\mathbf{m}_{\text{A}}$ does not perform well on this task due to its limited mathematical reasoning capabilities. On the other hand, the anchor model $\mathbf{m}_{\text{B}}$ does much better given its mathematical reasoning capabilities and transfer-learning from high-resource languages. Finally, we observe that $\mathbf{m}_{\text{A}\oplus\text{B}}$ outperforms both $\mathbf{m}_{\text{A}}$ and $\mathbf{m}_{\text{B}}$ on **18 of 25** low-resource and **9 of 10** high-resource languages, demonstrating effective composition of models. See Table 6 (Appendix A.2) for a complete set of evaluations. Note that the last row in Table 3 shows that $\mathbf{m}_{\text{B}}$ when fine-tuned on $\mathbf{D}_{\text{NTL}}$ leads to worse performance than the pre-trained $\mathbf{m}_{\text{B}}$ indicating forgetting. Composing domain-specific model $\mathbf{m}_{\text{A}}$ with $\mathbf{m}_{\text{B}}$ using CALM avoids this.

[1] We perform quality evaluations in Appendix 7.

Model	CC (P@1)	T2C (P@1)	C2T (chrF1)					
	HumanEval	MBPP	Python	PHP	Go	Java	JS	Ruby
PaLM2-XXS + Code ($\mathbf{m}_{\text{A}}$)	19.5	28.0	28.0	34.7	32.6	29.6	26.5	26.0
PaLM2-S ($\mathbf{m}_{\text{B}}$)	16.4	28.6	30.4	35.5	40.4	31.0	28.8	27.9
CALM ($\mathbf{m}_{\text{A}\oplus\text{B}}$)	**22.5**	**32.2**	**30.5**	**35.8**	**40.6**	**31.4**	**29.3**	**29.0**
$\mathbf{m}_{\text{B}}^{\text{Code}}$	24.3	43.0	18.9	35.0	41.1	31.1	20.2	27.6

Table 4: Evaluations for code generation and understanding across three tasks: Code Completion (CC), Text-to-Code (T2C), and Code-to-Text (C2T). Augmenting code understanding to $\mathbf{m}_{\text{B}}$ using $\mathbf{m}_{\text{A}}$ significantly improves performances across all datasets. $\mathbf{m}_{\text{B}}^{\text{Code}}$ represents a skyline where $\mathbf{m}_{\text{B}}$ further pretrained on the $\mathbf{D}_{\text{Code}}$, which shows catastrophic forgetting of text generation task.

4.3 Code Understanding and Generation

Code understanding and generation require two distinct types of capabilities: (**a**) knowledge of the syntax and semantics of code, and (**b**) knowledge of the world that the code is manipulating. While LLMs have a wealth of world knowledge, they could often lack the specific knowledge of code syntax due to a skewed representation of code data in their pretraining corpora. Conversely, small models trained specifically on code data could exhibit a good understanding of code syntax, but they may lack broad world knowledge and reasoning. CALM can enable best of both worlds.

Code Domain Data Here, we use the code-specific corpus, $\mathbf{D}_{\text{Code}}$, consisting of open-source code extracted from GitHub heads for a variety of programming languages to train $\mathbf{m}_{\text{A}}$.

Models Similar to §4.1, a version of the PaLM2-XXS model has been further pre-trained on $\mathbf{D}_{\text{Code}}$ is used as $\mathbf{m}_{\text{A}}$, while the base pre-trained PaLM2-S model acts as $\mathbf{m}_{\text{B}}$. We build $\mathbf{m}_{\text{A}\oplus\text{B}}$ by training CALM with only 7% of the same code data (data used for $\mathbf{m}_{\text{A}}$) to have a data parity.

Evaluation Tasks We evaluate the efficacy of CALM on three different tasks:

(i) Code-Completion (CC): Given an initial set of lines of a code, the model is prompted to complete the code snippet. Here the aim is to evaluate the model for code syntax. We perform zero-shot evaluations on HumanEval benchmark dataset (Chen et al., 2021) and report the Pass@1 (P@1) metric.

(ii) Text-to-Code (T2C): Given a textual context, the model is prompted to generate the corresponding code snippet. Here, the evaluation indicates language understanding and code generation capabilities. We perform 3-shot inference on the MBPP dataset (Austin et al., 2021) and report P@1.

(iii) Code-to-Text (C2T): Given a code snippet, the goal is to generate a natural language explanation of the code. This task evaluates code understanding and text generation. We perform 3-shot evaluations on the CodeXGlue benchmark (Lu et al., 2021) and report chrF1 scores across languages.

Results Table 4 reports comparative performance for the individual models $\mathbf{m}_{\text{A}}$ and $\mathbf{m}_{\text{B}}$, the composed version $\mathbf{m}_{\text{A}\oplus\text{B}}$, and a fine-tuned anchor baseline $\mathbf{m}_{\text{B}}^{\text{Code}}$. Firstly, evaluations on the HumanEval dataset suggest that $\mathbf{m}_{\text{A}}$ has a superior understanding of code syntax as a result of its additional training on $\mathbf{D}_{\text{Code}}$. While, due to the larger scale and general purpose pre-training of $\mathbf{m}_{\text{B}}$, it excels at general language understanding and hence performs better on the T2C and C2T tasks.

When employing CALM to compose the two models, we observe a clear transfer and composition of capabilities through significant performance improvements: 6.1% and 3.6% absolute gains over $\mathbf{m}_{\text{B}}$ on the CC and T2C tasks, respectively. We observe that fine-tuning $\mathbf{m}_{\text{B}}$ on $\mathbf{D}_{\text{Code}}$ leads to a significant decline in the C2T performance due to catastrophic forgetting. CALM retains the performance and is marginally better than $\mathbf{m}_{\text{B}}$ across all languages. We also study qualitative examples on the C2T task and observe interesting common patterns that are discussed in Appendix B.

		$\mathbf{m}_{\text{B}}^{\text{NTL/Code}}$	CALM $\mathbf{m}_{\text{A}\oplus\text{B}}$	Vanilla $\mathbf{m}_{\text{A}}$	Random $\mathbf{m}_{\text{A}}$	$\mathbf{m}_{\text{A}}$ as an encoder	LoRA
FLORES-200 (XX-En)	chrF1	**62.1**	60.5	59.2	58.8	59.3	59.2
	#($>\mathbf{m}_{\text{B}}$)	171	**175**	115	43	102	82
GSM-8K (LRL)	Accuracy	19.8	**21.4**	19.0	17.8	19.1	20.9
	#($>\mathbf{m}_{\text{B}}$)	15	**20**	15	9	12	15
GSM-8K (HRL)	Accuracy	27.1	**33.1**	29.7	28.5	29.1	31.2
	#($>\mathbf{m}_{\text{B}}$)	1	**11**	8	4	6	9
HumanEval	Pass@1	**24.3**	22.5	20.0	20.1	16.0	18.3
MBPP	Pass@1	**43.0**	32.2	28.0	27.0	27.0	28.7
CodeXGLUE	chrF1	29.0	**32.6**	32.2	32.1	32.0	**32.6**

Table 5: Comparative performance of CALM ($\mathbf{m}_{\text{A}\oplus\text{B}}$) across various possible ablations. The metric "#($>\mathbf{m}_{\text{B}}$)" depicts the number of languages for which the corresponding model is better than the base for NTL, $\mathbf{m}_{\text{B}}$—out of 192, 25, and 11 languages for the three tasks respectively. For all compared settings, the number of added parameters are kept the same.

4.4 Ablations

Influence of $\mathbf{m}_{\text{A}}$ We first study the influence of $\mathbf{m}_{\text{A}}$ by replacing it with vanilla and random variants during composition. Table 5 shows the variation of performance across NTL and Code tasks when the specialized $\mathbf{m}_{\text{A}}$ is replaced with a *vanilla* PaLM2-XXS checkpoint or an untrained version of the model, i.e., a *random* model. We see that there is a considerable drop of performance with these variants across all tasks. On FLORES-200 XX-En task, languages improved with composition drop to 115 and 43 with vanilla and random, respectively. A slight improvement of the vanilla model over $\mathbf{m}_{\text{B}}$ indicates that an un-specialized model (with a different training regime than $\mathbf{m}_{\text{B}}$) might have orthogonal capabilities leading to an enhanced model. This finding validates that performance gains seen with CALM is a result of utilizing $\mathbf{m}_{\text{A}}$ and not the added $\Theta_{\mathbf{C}}$ parameters.

Influence of iterative decoding We also investigate a variation where we use $\mathbf{m}_{\text{A}}$ as an encoder, i.e., an output token decoded at a given timestep is not amended to $\mathbf{m}_{\text{A}}$'s input. In this case, only the prefix representations of $\mathbf{m}_{\text{A}}$ are used. This setting eludes to past work for image and text models (Alayrac et al., 2022) where encoder and decoder models are composed. We observe a significant decline in performance across our various tasks when employing this setting.

Comparision with LoRA Finally, we evaluate a parameter efficient fine-tuning approach by training LoRA (Hu et al., 2022) layers to adapt $\mathbf{m}_{\text{B}}$. For all experiments, we set the LoRA rank such that the number of added parameters is equal to the number of parameters introduced with CALM. We also train LoRA on the same data as CALM, i.e., $\mathbf{D}_{\mathbf{C}}$. We see a considerable difference in performance between the two approaches across all tasks and metrics.

5 Conclusion

The proposed CALM framework composes an *anchor* LLM with specialized *augmenting* models to enable new tasks not achievable by either models individually. CALM does not require updating the individual models and learns a dense interaction between the models through a few trainable cross-attention parameters. Our experiments present consistent evidence that CALM learns to utilize the expertise from the two models. That is, when composed with relevant augmenting models, we observe a significant uptick in the anchor model's performance across multiple challenging tasks, such as low-resource translation, reasoning, and code explanation/generation.

That is, CALM is especially useful in scenarios where proprietary data and knowledge is stored in parametric models. With CALM, a foundational LLM could be augmented with such proprietary models to extend a variety of foundational capabilities such as reasoning, world knowledge, and coherent generation over the target proprietary domains. Finally, extensions of CALM could be used to acquire distinct knowledge from multiple augmenting models.

ACKNOWLEDGMENTS

This work was done during RB's pre-doctoral tenure at Google Research, India (GRI) with PT and PJ. RB is indebted to Manish Gupta, Divvy Thakkar, and all others who enabled this oppurtunity. RB would also like to thank the members of the Languages team and other researchers at GRI (and beyond), including the incredible pre-doctoral cohort. This work wouldn't have been possible without their constant support. Namely: Aishwarya P.S., Laurent El Shafey, and Qiao Zhang for their massive help in coding and debugging; Palak Jain and Sagar Gubbi for their feedback and support throughout the project; Kartikeya Badola, Shreyas Havaldar, Amandeep Kaur, and Rishabh Tiwari for being the first ears to all ideas; Cyrus Rashtchian and Richa Dixit for their mentorship.

REFERENCES

Jean-Baptiste Alayrac, Jeff Donahue, Pauline Luc, Antoine Miech, Iain Barr, Yana Hasson, Karel Lenc, Arthur Mensch, Katie Millican, Malcolm Reynolds, Roman Ring, Eliza Rutherford, Serkan Cabi, Tengda Han, Zhitao Gong, Sina Samangooei, Marianne Monteiro, Jacob Menick, Sebastian Borgeaud, Andrew Brock, Aida Nematzadeh, Sahand Sharifzadeh, Mikolaj Binkowski, Ricardo Barreira, Oriol Vinyals, Andrew Zisserman, and Karen Simonyan. Flamingo: a Visual Language Model for Few-Shot Learning, 2022. URL https://arxiv.org/abs/2204.14198.

Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, et al. Program synthesis with large language models. *ArXiv preprint*, abs/2108.07732, 2021. URL https://arxiv.org/abs/2108.07732.

Ankur Bapna, Isaac Caswell, Julia Kreutzer, Orhan Firat, Daan van Esch, Aditya Siddhant, Mengmeng Niu, Pallavi Baljekar, Xavier Garcia, Wolfgang Macherey, Theresa Breiner, Vera Axelrod, Jason Riesa, Yuan Cao, Mia Xu Chen, Klaus Macherey, Maxim Krikun, Pidong Wang, Alexander Gutkin, Apurva Shah, Yanping Huang, Zhifeng Chen, Yonghui Wu, and Macduff Hughes. Building machine translation systems for the next thousand languages, 2022.

Sébastien Bubeck, Varun Chandrasekaran, Ronen Eldan, Johannes Gehrke, Eric Horvitz, Ece Kamar, Peter Lee, Yin Tat Lee, Yuanzhi Li, Scott M. Lundberg, Harsha Nori, Hamid Palangi, Marco Túlio Ribeiro, and Yi Zhang. Sparks of artificial general intelligence: Early experiments with GPT-4. *ArXiv preprint*, abs/2303.12712, 2023. URL https://arxiv.org/abs/2303.12712.

Isaac Caswell, Theresa Breiner, Daan van Esch, and Ankur Bapna. Language ID in the wild: Unexpected challenges on the path to a thousand-language web text corpus. In *Proceedings of the 28th International Conference on Computational Linguistics*, pp. 6588–6608, Barcelona, Spain (Online), 2020. International Committee on Computational Linguistics. doi: 10.18653/v1/2020.coling-main.579. URL https://aclanthology.org/2020.coling-main.579.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *ArXiv preprint*, abs/2107.03374, 2021. URL https://arxiv.org/abs/2107.03374.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *ArXiv preprint*, abs/2110.14168, 2021. URL https://arxiv.org/abs/2110.14168.

Marta R. Costa-jussà, James Cross, Onur Çelebi, Maha Elbayad, Kenneth Heafield, Kevin Heffernan, Elahe Kalbassi, Janice Lam, Daniel Licht, Jean Maillard, Anna Sun, Skyler Wang, Guillaume Wenzek, Al Youngblood, Bapi Akula, Loïc Barrault, Gabriel Mejia Gonzalez, Prangthip Hansanti, John Hoffman, Semarley Jarrett, Kaushik Ram Sadagopan, Dirk Rowe, Shannon Spruit, Chau Tran, Pierre Andrews, Necip Fazil Ayan, Shruti Bhosale, Sergey Edunov, Angela Fan, Cynthia Gao, Vedanuj Goswami, Francisco Guzmán, Philipp Koehn, Alexandre Mourachko, Christophe Ropers, Safiyyah Saleem, Holger Schwenk, and Jeff Wang. No language left behind: Scaling human-centered machine translation. *ArXiv preprint*, abs/2207.04672, 2022. URL https://arxiv.org/abs/2207.04672.

Siddharth Dalmia, Dmytro Okhonko, Mike Lewis, Sergey Edunov, Shinji Watanabe, Florian Metze, Luke Zettlemoyer, and Abdelrahman Mohamed. LegoNN: Building Modular Encoder-Decoder Models, 2022. URL https://arxiv.org/abs/2206.03318.

Xavier Garcia, Aditya Siddhant, Orhan Firat, and Ankur Parikh. Harnessing multilinguality in unsupervised machine translation for rare languages. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pp. 1126–1137, Online, 2021. Association for Computational Linguistics. doi: 10.18653/v1/2021.naacl-main.89. URL https://aclanthology.org/2021.naacl-main.89.

Google, Rohan Anil, Andrew M. Dai, Orhan Firat, Melvin Johnson, Dmitry Lepikhin, Alexandre Passos, Siamak Shakeri, Emanuel Taropa, Paige Bailey, Zhifeng Chen, Eric Chu, Jonathan H. Clark, Laurent El Shafey, Yanping Huang, Katlorahy Meier-Hellstern, Gaurav Mishra, Erica Moreira, Mark Omernick, Kevin Robinson, Sebastian Ruder, Yi Tay, Kefan Xiao, Yuanzhong Xu, Yujing Zhang, Gustavo Hernandez Abrego, Junwhan Ahn, Jacob Austin, Paul Barham, Jan Botha, James Bradbury, Siddhartha Brahma, Kevin Brooks, Michele Catasta, Yong Cheng, Colin Cherry, Christopher A. Choquette-Choo, Aakanksha Chowdhery, Clément Crepy, Shachi Dave, Mostafa Dehghani, Sunipa Dev, Jacob Devlin, Mark Díaz, Nan Du, Ethan Dyer, Vlad Feinberg, Fangxiaoyu Feng, Vlad Fienber, Markus Freitag, Xavier Garcia, Sebastian Gehrmann, Lucas Gonzalez, Guy Gur-Ari, Steven Hand, Hadi Hashemi, Le Hou, Joshua Howland, Andrea Hu, Jeffrey Hui, Jeremy Hurwitz, Michael Isard, Abe Ittycheriah, Matthew Jagielski, Wenhao Jia, Kathleen Kenealy, Maxim Krikun, Sneha Kudugunta, Chang Lan, Katherine Lee, Benjamin Lee, Eric Li, Music Li, Wei Li, YaGuang Li, Jian Li, Hyeontaek Lim, Hanzhao Lin, Zhongtao Liu, Frederick Liu, Marcello Maggioni, Aroma Mahendru, Joshua Maynez, Vedant Misra, Maysam Moussalem, Zachary Nado, John Nham, Eric Ni, Andrew Nystrom, Alicia Parrish, Marie Pellat, Martin Polacek, Alex Polozov, Reiner Pope, Siyuan Qiao, Emily Reif, Bryan Richter, Parker Riley, Alex Castro Ros, Aurko Roy, Brennan Saeta, Rajkumar Samuel, Renee Shelby, Ambrose Slone, Daniel Smilkov, David R. So, Daniel Sohn, Simon Tokumine, Dasha Valter, Vijay Vasudevan, Kiran Vodrahalli, Xuezhi Wang, Pidong Wang, Zirui Wang, Tao Wang, John Wieting, Yuhuai Wu, Kelvin Xu, Yunhan Xu, Linting Xue, Pengcheng Yin, Jiahui Yu, Qiao Zhang, Steven Zheng, Ce Zheng, Weikang Zhou, Denny Zhou, Slav Petrov, and Yonghui Wu. Palm 2 technical report, 2023.

Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski, Bruna Morrone, Quentin de Laroussilhe, Andrea Gesmundo, Mona Attariyan, and Sylvain Gelly. Parameter-efficient transfer learning for NLP. In Kamalika Chaudhuri and Ruslan Salakhutdinov (eds.), *Proceedings of the 36th International Conference on Machine Learning, ICML 2019, 9-15 June 2019, Long Beach, California, USA*, volume 97 of *Proceedings of Machine Learning Research*, pp. 2790–2799. PMLR, 2019. URL http://proceedings.mlr.press/v97/houlsby19a.html.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. In *The Tenth International Conference on Learning Representations, ICLR 2022, Virtual Event, April 25-29, 2022*. OpenReview.net, 2022. URL https://openreview.net/forum?id=nZeVKeeFYf9.

Gabriel Ilharco, Marco Tulio Ribeiro, Mitchell Wortsman, Suchin Gururangan, Ludwig Schmidt, Hannaneh Hajishirzi, and Ali Farhadi. Editing models with task arithmetic. *ArXiv preprint*, abs/2212.04089, 2022. URL https://arxiv.org/abs/2212.04089.

Samuel Kessler, Bethan Thomas, and Salah Karout. An Adapter Based Pre-Training for Efficient and Scalable Self-Supervised Speech Representation Learning, 2021. URL https://arxiv.org/abs/2107.13530.

Aitor Lewkowycz, Anders Andreassen, David Dohan, Ethan Dyer, Henryk Michalewski, Vinay V. Ramasesh, Ambrose Slone, Cem Anil, Imanol Schlag, Theo Gutman-Solo, Yuhuai Wu, Behnam Neyshabur, Guy Gur-Ari, and Vedant Misra. Solving quantitative reasoning problems with language models. In *NeurIPS*, 2022. URL http://papers.nips.cc/paper_files/paper/2022/hash/18abbeef8cfe9203fdf9053c9c4fe191-Abstract-Conference.html.

Shuai Lu, Daya Guo, Shuo Ren, Junjie Huang, Alexey Svyatkovskiy, Ambrosio Blanco, Colin B. Clement, Dawn Drain, Daxin Jiang, Duyu Tang, Ge Li, Lidong Zhou, Linjun Shou, Long Zhou, Michele Tufano, Ming Gong, Ming Zhou, Nan Duan, Neel Sundaresan, Shao Kun Deng, Shengyu Fu, and Shujie Liu. Codexglue: A machine learning benchmark dataset for code understanding and generation. *ArXiv preprint*, abs/2102.04664, 2021. URL https://arxiv.org/abs/2102.04664.

Jiaqi Ma, Zhe Zhao, Jilin Chen, Ang Li, Lichan Hong, and Ed H. Chi. SNR: sub-network routing for flexible parameter sharing in multi-task learning. In *The Thirty-Third AAAI Conference on Artificial Intelligence, AAAI 2019, The Thirty-First Innovative Applications of Artificial Intelligence Conference, IAAI 2019, The Ninth AAAI Symposium on Educational Advances in Artificial Intelligence, EAAI 2019, Honolulu, Hawaii, USA, January 27 - February 1, 2019*, pp. 216–223. AAAI Press, 2019. doi: 10.1609/aaai.v33i01.3301216. URL https://doi.org/10.1609/aaai.v33i01.3301216.

Michael S Matena and Colin A Raffel. Merging models with fisher-weighted averaging. *Advances in Neural Information Processing Systems*, 35:17703–17716, 2022.

Mohammed Muqeeth, Haokun Liu, and Colin Raffel. Soft merging of experts with adaptive routing. *ArXiv preprint*, abs/2306.03745, 2023. URL https://arxiv.org/abs/2306.03745.

Jonas Pfeiffer, Aishwarya Kamath, Andreas Rücklé, Kyunghyun Cho, and Iryna Gurevych. AdapterFusion: Non-destructive task composition for transfer learning. In *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Main Volume*, pp. 487–503, Online, 2021. Association for Computational Linguistics. doi: 10.18653/v1/2021.eacl-main.39. URL https://aclanthology.org/2021.eacl-main.39.

Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *J. Mach. Learn. Res.*, 21:140:1–140:67, 2020. URL http://jmlr.org/papers/v21/20-074.html.

Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. *ArXiv preprint*, abs/2302.04761, 2023. URL https://arxiv.org/abs/2302.04761.

Yongliang Shen, Kaitao Song, Xu Tan, Dongsheng Li, Weiming Lu, and Yueting Zhuang. HuggingGPT: Solving AI Tasks with ChatGPT and its Friends in HuggingFace, 2023. URL https://arxiv.org/abs/2303.17580.

Freda Shi, Mirac Suzgun, Markus Freitag, Xuezhi Wang, Suraj Srivats, Soroush Vosoughi, Hyung Won Chung, Yi Tay, Sebastian Ruder, Denny Zhou, Dipanjan Das, and Jason Wei. Language models are multilingual chain-of-thought reasoners. In *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*. OpenReview.net, 2023. URL https://openreview.net/pdf?id=fR3wGCk-IXp.

Aditya Siddhant, Ankur Bapna, Orhan Firat, Yuan Cao, Mia Xu Chen, Isaac Caswell, and Xavier Garcia. Towards the next 1000 languages in multilingual machine translation: Exploring the synergy between supervised and self-supervised learning. *ArXiv preprint*, abs/2201.03110, 2022. URL https://arxiv.org/abs/2201.03110.

Karan Singhal, Tao Tu, Juraj Gottweis, Rory Sayres, Ellery Wulczyn, Le Hou, Kevin Clark, Stephen Pfohl, Heather Cole-Lewis, Darlene Neal, Mike Schaekermann, Amy Wang, Mohamed Amin, Sami Lachgar, Philip Andrew Mansfield, Sushant Prakash, Bradley Green, Ewa Dominowska, Blaise Agüera y Arcas, Nenad Tomasev, Yun Liu, Renee Wong, Christopher Semturs, S. Sara Mahdavi, Joelle K. Barral, Dale R. Webster, Gregory S. Corrado, Yossi Matias, Shekoofeh Azizi, Alan Karthikesalingam, and Vivek Natarajan. Towards expert-level medical question answering with large language models. *ArXiv preprint*, abs/2305.09617, 2023. URL https://arxiv.org/abs/2305.09617.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In Isabelle Guyon, Ulrike von Luxburg, Samy Bengio, Hanna M. Wallach, Rob Fergus, S. V. N. Vishwanathan, and Roman Garnett (eds.), *Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, December 4-9, 2017, Long Beach, CA, USA*, pp. 5998–6008, 2017. URL https://proceedings.neurips.cc/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html.

Ruize Wang, Duyu Tang, Nan Duan, Zhongyu Wei, Xuanjing Huang, Jianshu Ji, Guihong Cao, Daxin Jiang, and Ming Zhou. K-Adapter: Infusing Knowledge into Pre-Trained Models with Adapters. In *Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021*, pp. 1405–1418, Online, 2021. Association for Computational Linguistics. doi: 10.18653/v1/2021.findings-acl.121. URL https://aclanthology.org/2021.findings-acl.121.

Mitchell Wortsman, Gabriel Ilharco, Samir Yitzhak Gadre, Rebecca Roelofs, Raphael Gontijo Lopes, Ari S. Morcos, Hongseok Namkoong, Ali Farhadi, Yair Carmon, Simon Kornblith, and Ludwig Schmidt. Model soups: averaging weights of multiple fine-tuned models improves accuracy without increasing inference time. In Kamalika Chaudhuri, Stefanie Jegelka, Le Song, Csaba Szepesvári, Gang Niu, and Sivan Sabato (eds.), *International Conference on Machine Learning, ICML 2022, 17-23 July 2022, Baltimore, Maryland, USA*, volume 162 of *Proceedings of Machine Learning Research*, pp. 23965–23998. PMLR, 2022. URL https://proceedings.mlr.press/v162/wortsman22a.html.

Prateek Yadav, Derek Tam, Leshem Choshen, Colin Raffel, and Mohit Bansal. Resolving interference when merging models. *ArXiv preprint*, abs/2306.01708, 2023. URL https://arxiv.org/abs/2306.01708.

Andy Zeng, Maria Attarian, Brian Ichter, Krzysztof Choromanski, Adrian Wong, Stefan Welker, Federico Tombari, Aveek Purohit, Michael Ryoo, Vikas Sindhwani, Johnny Lee, Vincent Vanhoucke, and Pete Florence. Socratic Models: Composing Zero-Shot Multimodal Reasoning with Language, 2022. URL https://arxiv.org/abs/2204.00598.

A SUPPLEMENTARY MATERIAL FOR NTL

A.1 FLORES-200

Figure 2 depicts the gains over the anchor PaLM2-S model when augmented with a model that has been trained on $\mathbf{D}_{\text{NTL}}$. We see a positive gain through CALM for **175 of 192** languages. The highest gains are seen for low-resource languages since they are the most underrepresented in the original model. Diminishing returns with higher resource languages is seen and this trend is similar to the trend seen for $\mathbf{m}_{\text{B}}^{\text{NTL}}$.

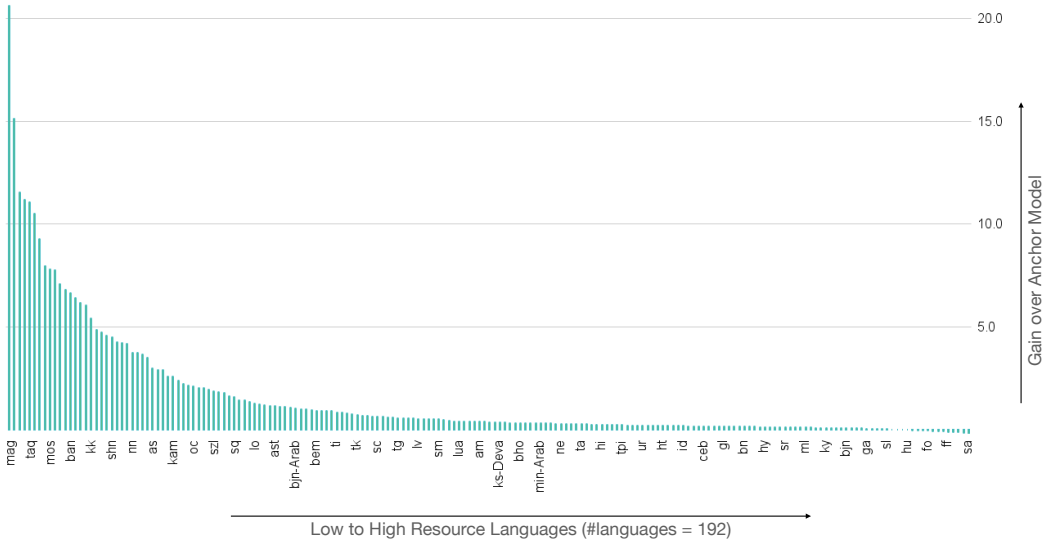

Figure 2: Gains seen by the composed model $\mathbf{m}_{\text{A}\oplus\text{B}}$ over the anchor model, $\mathbf{m}_{\text{B}}$, for the complete set of FLORES-200 languages. The languages are sorted from low to high-resource.

	$\mathbf{m}_{\text{A}}$	$\mathbf{m}_{\text{B}}$	$\mathbf{m}_{\text{A}\oplus\text{B}}$ (CALM)	$\mathbf{m}_{\text{B}}^{\text{NTL}}$
meo	7.6	28.8	**34.0**	33.2
mfa	4.0	14.0	**17.6**	20.4
pcm	4.4	**34.4**	33.6	31.6
efi	3.2	14.8	**18.0**	14.0
min	6.0	**25.2**	23.6	24.8
ilo	4.8	14.8	**16.8**	14.0
ady	6.4	30.0	**36.4**	29.2
mai	3.2	22.8	**24.8**	21.2
nso	6.0	**8.4**	**8.4**	9.6
mzn	4.8	31.6	**36.4**	27.6
bew	4.4	33.6	**34.8**	33.6
ts	4.8	7.2	**10.0**	11.6
dv	2.8	11.2	**14.8**	13.2

	$\mathbf{m}_{\text{A}}$	$\mathbf{m}_{\text{B}}$	$\mathbf{m}_{\text{A}\oplus\text{B}}$ (CALM)	$\mathbf{m}_{\text{B}}^{\text{NTL}}$
bho	4.0	23.6	**29.2**	22.8
cv	6.0	**17.6**	16.4	20.4
mni	3.6	2.8	**4.4**	6.0
or	2.4	9.6	**12.4**	12.0
kri	5.6	12.4	**18.8**	20.0
tk	5.2	27.2	**29.2**	28.8
gom	4.8	22.4	**25.2**	22.8
ug	6.0	23.2	**29.2**	26.4
ckb	3.2	25.6	**28.0**	27.2
as	1.2	5.2	**9.2**	4.0
doi	3.6	17.2	**22.4**	21.6
dz	**4.4**	0.8	0.4	0.0
avg.	4.5	18.6	**21.4**	19.8

Table 6: Performance evaluations on the complete set of low-resource languages for GSM-8K. Augmenting $\mathbf{m}_{\text{A}}$ with $\mathbf{m}_{\text{B}}$ as $\mathbf{m}_{\text{A}\oplus\text{B}}$ improves performance over $\mathbf{m}_{\text{B}}$ across a majority of languages. On average, we see an improvement of 2.8%.

	meo	mfa	pcm	efi	min	ilo	ady
Overlap	83.17	75.54	81.28	78.35	77.90	77.80	76.21
Delta	1.15	1.25	1.18	1.22	1.23	1.24	1.28
	mai	nso	mzn	bew	ts	dv	bho
Overlap	76.63	69.58	71.32	71.37	61.62	55.18	73.67
Delta	1.26	1.40	1.38	1.37	1.55	1.70	1.30
	cv	mni	or	kri	tk	gom	ug
Overlap	58.52	58.94	68.03	77.18	66.06	71.21	57.66
Delta	1.62	1.60	1.45	1.27	1.48	1.36	1.65

Table 7: Quality evaluation for the LRL GSM-8K dataset across languages. We created the dataset by translating the original English sentences of GSM-8K to the target language using the Google Translate API. We measure quality by back-translating the obtained examples back to English and measuring: (i) The ***overlap*** between the back-translated and the original English sentence, and (ii) The ***delta*** change in performance when PaLM2-S is evaluated on this back-translated version of GSM-8K as compared to the original version.

A.2 GSM-8K

Quality evaluation for LRL GSM-8K As described in Section 4.2, we created the GSM-8K dataset (Cobbe et al., 2021) for low-resource languages by using the Google Translate API to obtain silver translations in the target language from the source English sentence in the original dataset. We perform a quality evaluation of these examples by back-translating them back to English using the same translation API and defining two metrics over it:
(i) Overlap: The BLUE score measure between the actual example and the back-translated example,
(ii) Delta: The change in performance of the PaLM2-S model when evaluated on the original GSM-8K set as compared to the back-translated version.

Table 7 shows the values for these metrics across the various languages. We see that a decently high overlap value is seen across all languages. At the same time, the delta in performance is also minimal indicating that key attributes in the GSM-8K examples are not affected by translation.

Results on the complete language set Table 6 shows the comparative evaluations on the complete set of 25 low-resource languages for which GSM evaluations are performed. We see an improvement over the anchor model $\mathbf{m}_{\text{B}}$ for **20 of 25** languages. We also compare against the fully continued pre-trained version $\mathbf{m}_{\text{B}}^{\text{NTL}}$ and observe that $\mathbf{m}_{\text{A}\oplus\text{B}}$ outperform it for **18 of 25** languages.

B Qualitative Analysis

Table 8 depicts a few qualitative examples for the code-to-text, or the code explanation task, for Python. These examples depict examples for the three broader bucket of examples that we observe in cases when CALM yields the correct responses:

1. When neither of $\mathbf{m}_{\text{A}}$ or $\mathbf{m}_{\text{B}}$ generates the correct response but $\mathbf{m}_{\text{A}\oplus\text{B}}$ correctly attends over their latent representations to yield the correct output,
2. When either of $\mathbf{m}_{\text{A}}$ or $\mathbf{m}_{\text{B}}$ is seen to give the correct response while the other one is incorrect and $\mathbf{m}_{\text{A}\oplus\text{B}}$ generates the correct response that matches the generation from the correct model of $\mathbf{m}_{\text{A}}$ and $\mathbf{m}_{\text{B}}$, and
3. When both $\mathbf{m}_{\text{A}}$ and $\mathbf{m}_{\text{B}}$ generate the correct response and $\mathbf{m}_{\text{A}\oplus\text{B}}$ reproduces those generations.

We also observed similar qualitative patterns with other tasks for language inclusivity.

C Overhead with CALM

In this section, we include a detailed computation of the expected parametric and training overhead while composing given models using our proposed CALM framework.

`def ConsumeBool(self):`
`  try :`
`  result = ParseBool(self.token)`
`  except ValueError as e :`
`  raise self._ParseError(str(e))`
`  self.NextToken()`
`  return result`
	`def value(self):`
	`  if self.has_value:`
	`  return self._impl[OBJ].get_val(K)`
	`  else:`
	`  raise ValueError("Not found")`
	`  return`

⇒ Consumes a boolean	⇒ Print an error message and exit.
$\mathbf{m}_{\text{A}}$: Consumes a boolean	[a part of the given model prefix]
$\mathbf{m}_{\text{B}}$: The object is not a member	Exit with error message
CALM: Consumes a boolean	Print an error message and exit

`def get_positions(url):`
`  data = _get_resource(url)`
`  positions = [x for x in data['p']]`
`  return positions`
	`def distance(x0, y0, x1, y1):`
	`  return (`
	`  sqrt(pow(x1-x0,2) + pow(y1-y0,2)`
	`  )`

⇒ Returns a list of positions.	⇒ Returns the distance between two points
Positions of specified instruments.	Calculates the distance between two points
Get all positions.	Return the distance between two points
Returns a list of positions .	Calculates the distance between two points

Table 8: Cherry-picked qualitative examples for the code-to-text task on Python that depict examples that fall into a set of larger bucket of patterns that we observe across examples. CALM does well in various settings: (i) when $\mathbf{m}_{\text{A}}$produces the correct output but not $\mathbf{m}_{\text{B}}$, (ii) vice-versa—when $\mathbf{m}_{\text{B}}$ does well, and (iii) when neither of the two base models do well but a combination of intermediate representations allow the composed model to give the correct output. This shows that composition implicitly learns to do both: routing across models and a combination, based on a given input.

C.1 Parametric Overhead

Building from the notations in §3.1, let's say the two models $\mathbf{m}_{\text{A}}$ and $\mathbf{m}_{\text{B}}$ have N_A and N_B number of standard transformer layers, respectively, with each layer of output dimensionality D_A and D_B. As mentioned, we choose $n = n_A = n_B$ number of layers to perform the composition.

$$\text{\# Parameters for each } f_{\text{proj}} \text{ layer} = (D_A * D_B)$$
$$\text{\# Parameters for each } f_{\text{cross}} \text{ layer} = (3 * D_B^2)$$
$$\text{\# Parameters added during composition} = n * (D_A * D_B + 3 * D_B^2)$$
$$\text{\# Parameters in } \mathbf{m}_{\text{B}} = N_B * (V_B * D_B + 3 * D_B^2 + 2 * D_B * D_B * K_B)$$

where, V_B and K_B depict the vocabulary size and hidden multiplication factor, respectively.

Let's consider some standard transformer configurations to understand the parameter overhead. As an example, consider the layer configurations of standard BERT models: BERT-small ($\mathbf{m}_{\text{A}}$) and BERT-large ($\mathbf{m}_{\text{B}}$). In this case: N_A = 4, D_A = 512, N_B = 24, D_B = 1024, V_B = 30K, K_B = 4. Assuming that we select all layers of $\mathbf{m}_{\text{B}}$, the value of n = 4. Hence,

$$\text{\# Parameters added during composition} = 4 * (512 * 1024 + 3 * 1024^2) \approx 1.5 \times 10^7 \approx 15\text{M}$$
$$\text{\# Parameters in } \mathbf{m}_{\text{B}} = 24 * (30\text{K} * 1024 + 3 * 1024^2 + 2 * 1024^2 * 4) \approx 1\text{B}$$
$$\text{\%age of new parameters added} = 15\text{M} * 100/1\text{B} = 1.5\%$$

Hence, number of parameters added during composition ≈ 1.5% of those in $\mathbf{m}_{\text{B}}$.

C.2 Training Overhead

While back propagation over $\mathbf{m}_{\text{B}}$ is indeed required while training CALM, the total training costs are still significantly lesser than training $\mathbf{m}_{\text{B}}$, owing to the training examples/iterations required.

Firstly, as discussed above, the additional number of parameters introduced during composition is 1.5% of the number of parameters of $\mathbf{m}_{\text{B}}$—hence, a negligible parametric addition.

Further, since only 5-7% of the total $\mathbf{m_B}$ fine-tuning data is required to train CALM, the training cost of CALM is minimal with respect to training cost of training the entire anchor model.

Moreover, since our experiments consider an $\mathbf{m_A}$ that has 5-20% of parameters as $\mathbf{m_B}$, even the net cost of training $\mathbf{m_A}$ and CALM is significantly lesser than training $\mathbf{m_B}$.

Let's assume that (i) the cost of fine-tuning $\mathbf{m_B}$ on the complete data is X, (ii) number of parameters in $\mathbf{m_A}$ is 10% of those in $\mathbf{m_B}$, and (iii) the amount of data required to train CALM is 2% of $\mathbf{m_B}$ training. Assuming a linear scaling factor of training cost (FLOPS) with model parameters and data:

Cost of training CALM $\approx 0.02 \times X$ **= 2% of** $\mathbf{m_B}$ **training.**
Cost of training $\mathbf{m_A}$ **+ CALM** $\approx (0.10 * X + 0.02 * X) = 0.12 \times X$ **= 12% of** $\mathbf{m_B}$ **training.**